

# Sesión 4

## Streaming e inferencia en tiempo real



Cuando los datos no llegan de a uno sino como un flujo continuo, sin principio ni final.

# Agenda de hoy

|        |                                                                                     |                                                                                            |
|--------|-------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| 0 : 00 |    | <b>Recap y encuadre</b><br>Dónde estamos en la plataforma                                  |
| 0 : 10 |    | <b>Datos acotados vs ilimitados</b><br>Batch vs stream · datos en reposo vs en movimiento  |
| 0 : 35 |    | <b>Almacenar el flujo</b><br>Mensajería directa vs brokers · Kafka (logs, topics)          |
| 1 : 10 |    | <b>Procesar el flujo</b><br>Filtrar, enriquecer, agregar, detectar eventos                 |
| 1 : 30 |    | <b>Tiempo, ventanas y estado</b><br>Event vs processing time · tumbling/sliding · stateful |
| 1 : 55 |   | <b>Streaming en MLOps</b><br>Inferencia online y monitoreo por ventana                     |
| 2 : 10 |  | <b>Práctica</b><br>Productor → topic → consumidor que puntúa el modelo                     |
| 2 : 40 |  | <b>Mini-TP 4 · Hito TP #1 · cierre</b><br>Consumidor propio · equipos + arquitectura       |

## La plataforma ya responde pedidos; hoy reacciona a un flujo continuo

REST

GraphQL

gRPC

Streaming

Cloud/Lake

Federado

Seguridad

Integra

### En las Sesiones 1–3

- REST y GraphQL: el borde externo
- gRPC: comunicación interna de baja latencia
- Siempre bajo un modelo request→response

### Hoy resolvemos

- Los datos llegan solos, sin fin: eventos
- Hay que reaccionar en el momento, no esperar al lote
- Puntuar el modelo sobre el flujo → streaming

# Datos acotados vs ilimitados



## Bounded (acotados)

Finitos, con principio y fin. Se guardan y se procesan por lotes (batch).

- Un CSV, una tabla, un dataset histórico
- Se procesan cuando alguien lo pide
- Alta latencia, alto throughput



## Unbounded (ilimitados)

Infinitos, sin principio ni fin discretos. Una serie continua de eventos.

- Transacciones, clics, sensores IoT
- Se procesan a medida que llegan
- Baja latencia: reaccionar ya

# Tres formas de procesar datos ilimitados



## Batch de almacenados

Se guarda todo y se procesa a intervalos fijos (cada noche). Simple, pero el dato envejece.



## Micro-batch

Se procesan lotes pequeños y frecuentes (cada pocos segundos). Un punto medio.



## Streaming continuo

Cada evento se procesa al llegar. Latencia mínima; el resultado siempre está fresco.

*Hoy nos centramos en el streaming continuo: incorporar nuevos datos sin parar para recalcular un resultado.*

# El cambio de mentalidad

## DBMS: datos en reposo

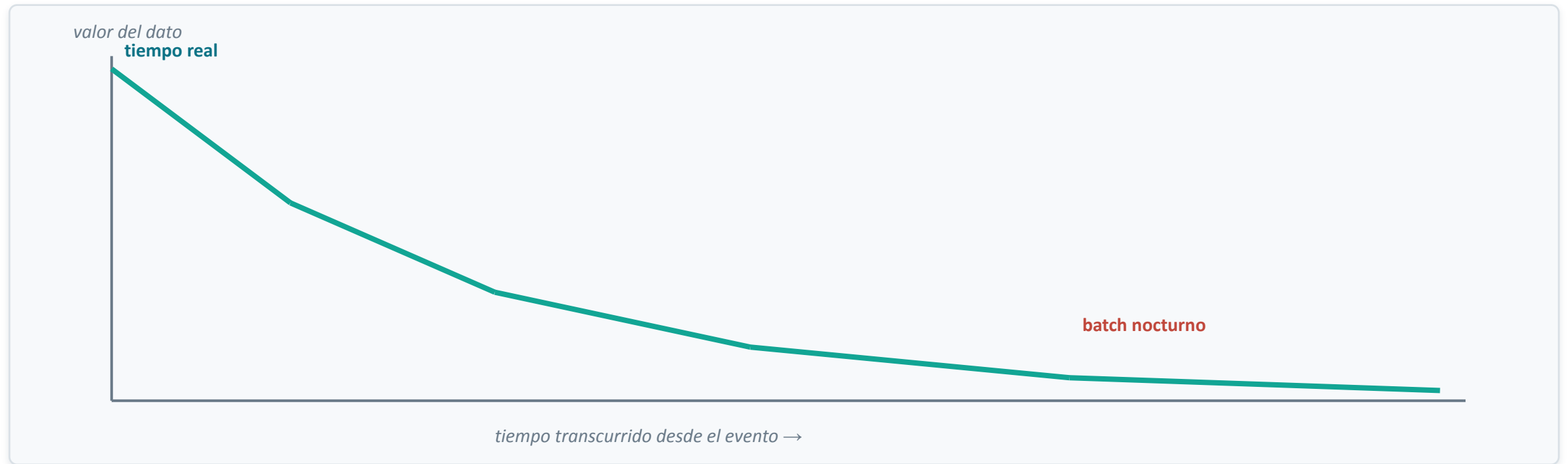
- Primero se almacena e indexa el dato
- Se procesa solo cuando el usuario consulta
- La consulta es efímera, el dato es persistente
- 'Pull': tú vas a buscar la respuesta

## Stream: datos en movimiento

- La consulta es persistente (siempre corriendo)
- El dato pasa a través de ella una sola vez
- Reacciona a cada evento al llegar
- 'Push': el resultado te llega a ti

# El valor de muchos datos decae con el tiempo

Para muchas decisiones (fraude, alertas, recomendación en vivo) el valor de un dato es máximo apenas ocurre y cae rápido. Actuar tarde vale menos.



# Características de un flujo de datos



## Registros pequeños

Cada evento es chico (un clic, una transacción, una lectura).



## Volumen muy alto

Pero llegan muchísimos: el volumen agregado es enorme.



## Ritmo irregular

Ráfagas y períodos tranquilos: la carga no es constante.



## Pueden llegar tarde

Y desordenados: hay que lidiar con el tiempo explícitamente.

# ¿Cómo llevamos eventos de productores a consumidores?



## Mensajería directa

El productor envía el evento directo al consumidor. Mínima latencia (cirugía remota, trading).

### El problema:

- Si un consumidor se cae, pierde eventos (no durable)
- Si el productor va más rápido, hay que descartar o frenar (backpressure)



## Broker de mensajes

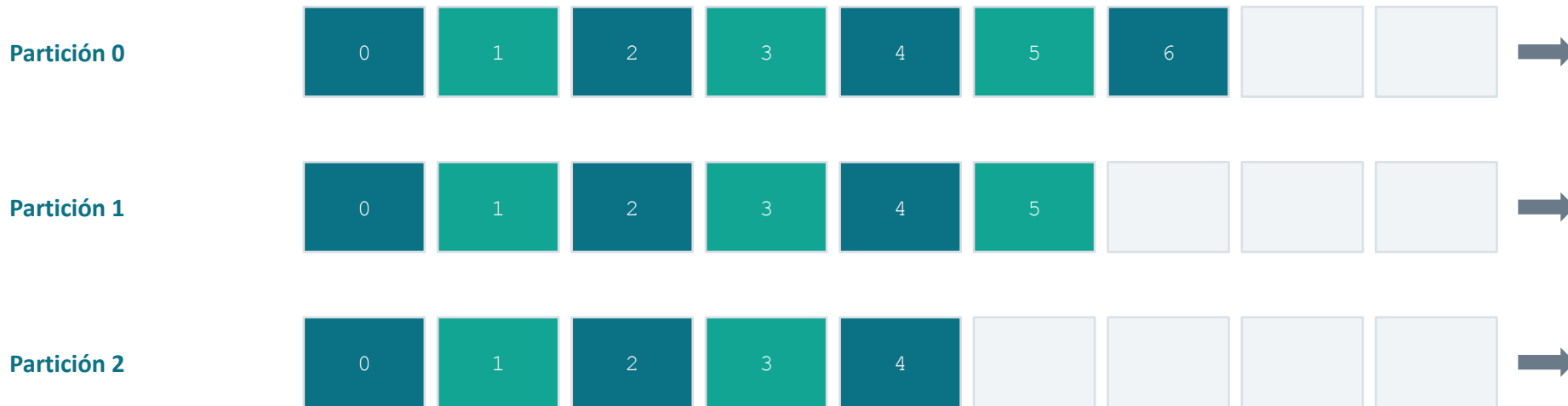
Un intermediario recibe y guarda los eventos; los consumidores leen a su ritmo.

### Dos modos de reparto:

- Load balancing: cada mensaje a UN consumidor
- Fan-out: cada mensaje a TODOS los consumidores

# Un broker basado en logs: topics, particiones y offsets

Kafka es un servicio de registros (logs) distribuido, particionado y replicado. Un topic es un flujo de mensajes; se divide en particiones que se leen y escriben independientemente.



**Append-only e inmutable:** solo se agrega al final. Ordenado dentro de una partición. Cada consumidor lleva su propio offset y lee a su ritmo.

# Garantías y piezas de la arquitectura



## Orden por partición

El orden se garantiza dentro de una partición, no entre particiones.



## Entrega at-least-once

Un mensaje puede reprocesarse: los consumidores deben ser idempotentes.



## Brokers sin estado

El broker guarda los mensajes; el consumidor lleva su offset. Los mensajes caducan por política (retención).



## Grupos de consumidores

Un grupo reparte las particiones entre sus miembros (escala horizontal).

*Alternativas de la misma familia: Redpanda (compatible con la API de Kafka), AWS Kinesis, Google Pub/Sub.*

# Las operaciones básicas



## Filtrar

Descartar eventos irrelevantes antes de seguir (reduce volumen).



## Enriquecer

Unir el evento con datos de una DB o API (features, contexto).



## Agregar

Resumir por ventanas de tiempo: conteos, promedios, máximos.



## Detectar eventos

Reconocer patrones y disparar una acción cuando se cumplen.

*En MLOps, 'enriquecer + puntuar el modelo' sobre cada evento es la inferencia online.*

# Tiempo del evento vs tiempo de procesamiento



## Event time

Cuándo ocurrió el evento de verdad (marca de tiempo puesta en el origen). Es el tiempo 'correcto' para agregar.



## Processing time

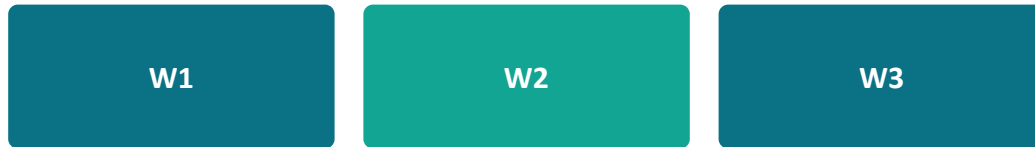
Cuándo el sistema recibió el evento. Es más simple, pero desordena si hay demoras de red.

**Watermark (marca de agua):** una señal en el flujo que declara "ya llegó todo lo anterior al tiempo  $t$ ". Permite cerrar ventanas y tolerar eventos que llegan tarde o desordenados.

# Ventanas: tumbling vs sliding

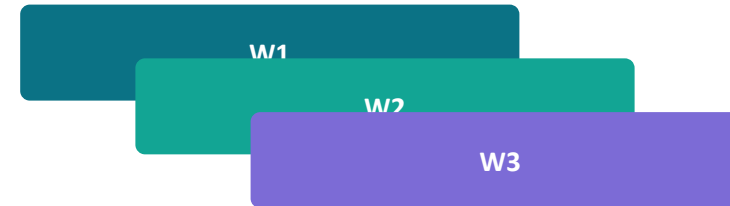
Como el flujo es infinito, agregamos sobre ventanas: buffers que retienen eventos por conteo o por tiempo.

## Tumbling (fijas, no se solapan)



Cada evento cae en exactamente una ventana. Ideal para reportes por bloque (cada 5 min).

## Sliding (deslizantes, se solapan)



La ventana avanza de a pasos; un evento cae en varias. Ideal para métricas móviles (media de los últimos 5 min, cada minuto).

**Stateless vs stateful.** Filtrar o transformar un evento es sin estado (fácil de paralelizar). Agregar por ventana, detectar patrones o contar es con estado: el operador recuerda las tuplas recibidas.

# Para qué lo usamos



## Inferencia online

Puntuar el modelo sobre cada evento al llegar (fraude, recomendación en vivo).



## Features en tiempo real

Calcular features por ventana (gasto en la última hora) para alimentar al modelo.



## Monitoreo por ventana

Vigilar throughput, latencia p95 y drift de entradas de forma continua.



## Reentrenar por eventos

El flujo alimenta el pipeline: datos frescos disparan reentrenamientos.

*El modelo que servimos por REST/gRPC ahora también se aplica sobre un flujo: misma pieza, nuevo modo de consumo.*

# Qué construimos

Un pipeline de scoring en streaming: un productor emite eventos, un consumidor puntúa el modelo sobre cada uno y agrega métricas por ventana. Primero en memoria; después con Kafka en Docker.



**NOTEBOOK** `streaming_tutorial.ipynb`

# Streaming en memoria: productor, consumidor y ventana

```
import queue, threading, time, random
stream = queue.Queue()

def productor(): # emite eventos sin fin
    while True:
        stream.put({'values':[random.gauss(0,1)
                               for _ in range(3)], 't':time.time()})
        time.sleep(0.01)

def consumidor(): # puntua el modelo online
    while True:
        ev = stream.get()
        y = modelo.predict([ev['values']])[0]
        ventana.append((ev['t'], y)) # para agregar
```

## Qué mirar

- El flujo es infinito: un while sin fin
- El consumidor puntúa al llegar (online)
- La ventana retiene eventos recientes
- Sobre ella: throughput, p95, drift

**NOTEBOOK** streaming\_tutorial.ipynb

## Lo mismo con Kafka (Redpanda) en Docker

```
# 1) levantar un broker compatible con Kafka
docker run -d --name redpanda -p 9092:9092 \
  redpandadata/redpanda redpanda start \
  --overprovisioned --smp 1 --check=false
# 2) dependencias
uv add kafka-python
```

```
from kafka import KafkaProducer, KafkaConsumer
prod = KafkaProducer(bootstrap_servers='localhost:9092')
prod.send('eventos', b'{...}') # productor
# consumer.subscribe(['eventos']) -> puntuar cada mensaje
```

**NOTEBOOK** streaming\_tutorial.ipynb

### Qué mirar

- El topic 'eventos' es el log particionado
- El productor escribe; el consumer lee por offset
- Mismo código de scoring que en memoria
- Redpanda = API de Kafka, más liviano

## Errores comunes (y cómo evitarlos)



### Confundir event y processing time

Agrega por el tiempo del evento, no por cuándo llegó.



### Ventanas sin cierre

Sin watermark, las ventanas nunca cierran o pierden tardíos.



### Estado que crece sin límite

Un operador stateful debe expirar lo viejo (retención).



### Consumidor no idempotente

At-least-once reprocesa: evita efectos dobles.



### Ignorar el backpressure

Si el consumidor no da abasto, hay que frenar o escalar.



### Cargar el modelo por evento

Cárgalo una vez; puntúa millones de eventos con él.



MINI-TP 4 · INDIVIDUAL · ENTREGA ESTA SEMANA

## Puntúa tu modelo sobre un flujo

### Consigna

- Simula (o consume de Kafka) un flujo de eventos con las features de tu modelo.
- Escribe un consumidor que puntúe tu modelo sobre cada evento (inferencia online).
- Agrega por ventana: throughput (ev/s), latencia p95 y un indicador simple de drift de entradas.
- Dispara una alerta cuando una métrica cruce un umbral.
- Compara el resultado contra puntuar el mismo lote en batch.

### Se evalúa

- Corre de punta a punta
- Inferencia online sobre el flujo
- Métricas por ventana correctas
- Reflexión streaming vs batch

**Actividad:** `mini_tp4_actividad.ipynb` (starter con celdas TODO).

# Equipos y arquitectura de referencia

Hoy arranca el TP integrador grupal. Este primer hito no pide código: pide decidir el equipo y dibujar la arquitectura de la plataforma que van a construir sobre las capas de este bimestre.



## Qué entregar

- Equipo (grupos) y roles
- Caso/dataset elegido y objetivo del servicio
- Diagrama de arquitectura de referencia
- Qué capa cubre cada sesión (REST, GraphQL, gRPC, streaming, Data Lake, seguridad)
- Plan de hitos hasta la defensa



## Arquitectura base

Airflow + MLflow + FastAPI + MinIO/S3 sobre Docker. Cada sesión agrega una capa sobre el mismo modelo. El integrador viene medio construido con los mini-TP.

*Próximos hitos: checkpoint evaluado en la Sesión 6 · defensa en la Sesión 8.*

# Dónde quedó la plataforma

REST

GraphQL

gRPC

Streaming

Cloud/Lake

Federado

Seguridad

Integra

## Hoy logramos

- Distinguimos batch de streaming
- Kafka: topics, particiones, offsets, ventanas
- Puntuamos el modelo sobre un flujo (online)



## Próxima · Sesión 5

Nube y Data Lakes: dónde aterriza todo ese flujo de eventos para almacenarse barato y a escala (MinIO/S3), y cómo se consume después.

*Para la próxima: Mini-TP 4 corriendo y el Hito #1 (equipos + arquitectura) entregado.*

# Referencias y recursos

## Herramientas y código

- Apache Kafka — [kafka.apache.org/documentation](https://kafka.apache.org/documentation) · Redpanda — [docs.redpanda.com](https://docs.redpanda.com) · paquetes: `uv add kafka-python`
- Repositorio del curso: `clase4/Practica` (`streaming_tutorial.ipynb`, `mini_tp4_actividad.ipynb`).

## Lectura

- Designing Data-Intensive Applications — Martin Kleppmann (O'Reilly): cap. 11, streams y logs.
- Designing Machine Learning Systems — Chip Huyen (O'Reilly): batch vs streaming features, online prediction.